

LEOS USER REPO API

API documentation for LEOS User Repo

Version: 5.5.0-SNAPSHOT

1. App Info

1.1 GET /

Overview: Get application information

Description: Retrieves application version and build information

Controller: [App Info](#)

Method: `getAppInfo`

Responses

Status Code	Description	Content-Type	Schema
200	Application information retrieved successfully	application/json	Map
500	Internal server error	application/json	object

2. Entity Management

2.1 GET /entities/{org}/users

Overview: Search users by organization and key

Description: Search users by organization and key

Controller: [Entity Management](#)

Method: `searchUsersByOrganizationAndKey`

Parameters

Name	In	Required	Type	Description
org	path	Yes	string	
searchKey	query	Yes	string	

Responses

Status Code	Description	Content-Type	Schema
200	Users found successfully	application/json	Collection

500	Internal server error	application/json	UserRepoExceptionResponse
-----	-----------------------	------------------	---------------------------

2.2 GET/entities

Overview: Get all organizations

Description: Retrieves list of all organizations

Controller: [Entity Management](#)

Method: [getAllOrganizations](#)

Responses

Status Code	Description	Content-Type	Schema
200	Organizations retrieved successfully	application/json	Collection
500	Internal server error	application/json	UserRepoExceptionResponse

2.3 POST/entities

Overview: Create special entity

Description: Creates a special entity

Controller: [Entity Management](#)

Method: [createEntity](#)

Request Body

Required	Content-Type	Schema
Yes	application/json	EntityDto

Responses

Status Code	Description	Content-Type	Schema
200	Special entity created successfully	application/json	EntityDto
400	Invalid DTO	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

2.4 PATCH/entities

Overview: Update special entity

Description: Updates a special entity

Controller: [Entity Management](#)

Method: `updateEntity`

Request Body

Required	Content-Type	Schema
Yes	application/json	EntityDto

Responses

Status Code	Description	Content-Type	Schema
200	Special entity updated successfully	application/json	EntityDto
400	Special entity not found or invalid DTO	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

2.5 GET/entities/special

Overview: Get special entities

Description: Retrieves all special entities for a given organization or all special enti...

Controller: [Entity Management](#)

Method: `special`

Parameters

Name	In	Required	Type	Description
orgName	query	No	string	- If orgName is null or empty, then all SpecialEntity records are returned; - Matching special entities + the organization itself are returned otherwise;

Responses

Status Code	Description	Content-Type	Schema
200	Special entities retrieved successfully	application/json	Collection
500	Internal server error	application/json	UserRepoExceptionResponse

2.6 DELETE/entities/{id}

Overview: Delete special entity

Description: Deletes a special entity

Controller: [Entity Management](#)

Method: `deleteEntity`

Parameters

Name	In	Required	Type	Description
id	path	Yes	string	

Responses

Status Code	Description	Content-Type	Schema
200	Special entity deleted successfully		
400	Special entity with that ID was not found	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

2.7 GET/entities/{userId}

Overview: Get entities for user

Description: Retrieves all full path entities for a user

Controller: [Entity Management](#)

Method: `getAllFullPathEntitiesForUser`

Parameters

Name	In	Required	Type	Description
userId	path	Yes	string	

Responses

Status Code	Description	Content-Type	Schema
200	Entities retrieved successfully	application/json	Collection
500	Internal server error	application/json	UserRepoExceptionResponse

3. User Management

3.1 GET /users/search

Overview: Search for users by term and/or entity

Description: Retrieves special user by login

Controller: [User Management](#)

Method: `searchUsers`

Parameters

Name	In	Required	Type	Description
searchKey	query	No	string	Search key matching First Name, Last Name, Username, Email
entityId	query	No	string	If passed, will look for users only belonging to the given entity

Responses

Status Code	Description	Content-Type	Schema
200	Users search OK	application/json	Page
500	Internal server error	application/json	UserRepoExceptionResponse

3.2 GET /users

Overview: Search users

Description: Search users by key with optional context and reference

Controller: [User Management](#)

Method: `searchUsers`

Parameters

Name	In	Required	Type	Description
searchKey	query	Yes	string	Search key
searchContext	query	No	string	Search context
searchReference	query	No	string	Search reference

Responses

Status Code	Description	Content-Type	Schema
200	Users found successfully	application/json	Collection

500	Internal server error	application/json	UserRepoExceptionResponse
-----	-----------------------	------------------	---------------------------

3.3 POST/users

Overview: Create user

Description: Creates a new user

Controller: [User Management](#)

Method: [createUser](#)

Request Body

Required	Content-Type	Schema
Yes	application/json	UserDto

Responses

Status Code	Description	Content-Type	Schema
200	User created successfully	application/json	UserDto
400	On: - Username conflict; - Any of the passed entities does not exist;	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

3.4 PATCH/users

Overview: Update user

Description: Updates an existing user

Controller: [User Management](#)

Method: [updateUser](#)

Request Body

Required	Content-Type	Schema
Yes	application/json	UserDto

Responses

Status Code	Description	Content-Type	Schema
200	User updated successfully	application/json	UserDto
400	User not found or invalid DTO object	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

3.5 GET/users/jobTitle/{jobTitle}

Overview: Get users by job title

Description: Retrieves users with specified job title

Controller: [User Management](#)

Method: `getUsersForJobTitle`

Parameters

Name	In	Required	Type	Description
jobTitle	path	Yes	string	Job Title

Responses

Status Code	Description	Content-Type	Schema
200	Users found successfully	application/json	Collection
500	Internal server error	application/json	UserRepoExceptionResponse

3.6 POST/users/connectedEntity

Overview: Add special entity for user

Description: Adds a special entity connection for a user

Controller: [User Management](#)

Method: `addSpecialEntityForUser`

Request Body

Required	Content-Type	Schema
Yes	application/json	SpecialEntityRequest

Responses

Status Code	Description	Content-Type	Schema
200	Special entity added successfully	application/json	boolean
500	Internal server error	application/json	UserRepoExceptionResponse

3.7 DELETE/users/{userLogin}

Overview: Delete user

Description: Deletes a user by login

Controller: [User Management](#)

Method: `deleteUser`

Parameters

Name	In	Required	Type	Description
userLogin	path	Yes	string	User login

Responses

Status Code	Description	Content-Type	Schema
204	User deleted successfully		
400	User not found	application/json	UserRepoExceptionResponse
500	Internal server error	application/json	UserRepoExceptionResponse

3.8 GET/users/{userId}

Overview: Get user by ID

Description: Retrieves user details by user ID

Controller: [User Management](#)

Method: `getUser`

Parameters

Name	In	Required	Type	Description
userId	path	Yes	string	User ID

Responses

Status Code	Description	Content-Type	Schema
200	User found successfully	application/json	User
500	Internal server error	application/json	UserRepoExceptionResponse